DashVMC

Real-Time Discrete World Model Control in Geometry Dash

Contributors
Florent Tariolle[1], Florian Yger[1,2]

[1]**INSA Rouen Normandy** [2]**LITIS**

World-model agents are usually evaluated in simulators that can wait for the policy; live games impose the opposite constraint, requiring capture, prediction, and action before the next frame. We present *DashVMC*, which learns a compact, action-conditioned world model from approximately two hours of recorded Geometry Dash gameplay. To test whether the learned dynamics are actionable, a controller is initialized by behavioural cloning (BC) and refined with Proximal Policy Optimization (PPO) entirely in frozen-model rollouts, without further interaction with the live game. Across three controller seeds, the refined policies survive longer than their BC initializations on all three official levels and a held-out community layout. At deployment, the baseline skips visual generation and sustains a 60-Hz capture-to-action loop on a consumer GPU. Action-conditioned continuations and rollout diagnostics show that the model remains useful for control despite imperfect long-horizon fidelity.

Project page: tariolle.github.io/dash-vmc
Code: github.com/Tariolle/dash-vmc
Correspondence: florent.tariolle@insa-rouen.fr

1 Introduction

A world model learns transition dynamics $p_\theta(s_{t+1} \mid s_t, a_t)$ and can be rolled forward so that an agent practices without further environment interaction (LeCun, 2022; Ha and Schmidhuber, 2018). DashVMC asks whether such practice can improve a policy that must then act in a live, non-paused game. It predicts a 64-token next grid in parallel for inexpensive imagined rollouts. The baseline deployment skips generation and reuses the transformer's temporal summary; a matched ablation tests whether the transformer must remain in the live loop.

Most world-model agents are evaluated in synchronous environments that wait for the policy. A live, non-paused application reverses that contract: screen capture, perception, inference, and action dispatch must finish before the action becomes stale.

Geometry Dash is a small but unforgiving test of this contract. The avatar moves continuously, the action space is binary, and a mistimed jump causes immediate death. Levels are deterministic, but the agent observes only captured pixels and acts through keyboard events; it does not receive emulator state during control. The apparent simplicity therefore isolates a demanding question: can an agent learn when to act from pixels, improve without touching the live game, and still react within narrow timing windows on modest hardware?

DashVMC follows the perception–dynamics–control organization of World Models (Ha and Schmidhuber, 2018): it compresses each frame into a discrete grid, learns action-conditioned grid dynamics, initializes a lightweight action model with BC, and refines it with PPO (Schulman et al., 2017) in the frozen model.

Our contributions are:

1. a compact discrete, action-conditioned world model learned from approximately two hours of offline gameplay, whose rollouts support policy refinement without further live-game interaction;
2. a decoder-free baseline that sustains a 60-Hz capture-to-action loop on a consumer GPU, together with

a spatial-only variant that removes the transformer at inference without a consistent live-performance deficit; and

3. evidence that the learned dynamics are actionable: controllers refined only in frozen-model rollouts improve over their exact BC parents on all three official levels across three seeds and on a held-out layout.

Diagnostics cover action sensitivity, rollout fidelity, controller input, dynamics losses, and latency.

2 Related work

World models and imagination. World Models demonstrated that a compact controller optimized in learned dreams can transfer to the real environment (Ha and Schmidhuber, 2018). IRIS combines a discrete autoencoder with an autoregressive transformer for data-efficient Atari learning (Micheli et al., 2023); Δ-IRIS reduces sequence cost with separate change and context tokens (Micheli et al., 2024); DIAMOND uses diffusion for imagination-trained control (Alonso et al., 2024); and DreamerV3 scales categorical recurrent latents across domains (Hafner et al., 2025). TWISTER adds action-conditioned CPC to transformer dynamics (Burchi and Timofte, 2025), which DashVMC adopts alongside parallel spatial prediction. Earlier screenshot-based Geometry Dash control used DQN and imitation under a fabricated timestep (Li and Rafferty, 2017); here the game clock continues during capture and inference.

FSQ tokenization and policy initialization. FSQ replaces learned vector-quantization codebooks with bounded scalar levels, avoiding commitment losses and codebook collapse while exposing an integer coordinate lattice (Mentzer et al., 2024); we use the improved iFSQ bounding function (Lin et al., 2026). Demonstration-based RL can reduce costly early exploration (Hester et al., 2018; Baker et al., 2022); here recorded actions initialize the controller, while the retained policy is selected only after imagined PPO refinement. Deterministic Sobel preprocessing removes appearance detail before tokenization, providing a compact task-specific alternative to richer continuous or generative representations.

3 DashVMC

DashVMC is organized around a simple loop: recorded play trains a world model, the action model is refined in rollouts from the frozen dynamics model, and the resulting controller is returned to the live game. The visual encoder, dynamics model, and action model are trained in sequence and then frozen for deployment. Figure 1 shows how the same learned state supports both imagined practice and live reaction.

3.1 Visual tokenization

Each RGB capture is cropped, converted to grayscale, filtered with Sobel edges, and resized to 64×64. A convolutional autoencoder maps the result through three stride-2 stages to a four-channel 8×8 latent map. Each spatial vector is independently quantized with iFSQ levels $[8, 5, 5, 5]$, using the bounding function $2\,\mathrm{sigmoid}(1.6z) - 1$, and yields 64 token IDs from 1000 implicit codes (Mentzer et al., 2024; Lin et al., 2026). The tokenizer is trained on adjacent frames with reconstruction, temporal-slowness, and latent-uniformity objectives (Xia et al., 2025). Once selected, its encoder is frozen; its decoder is used to visualize dreams but is not required by the action model.

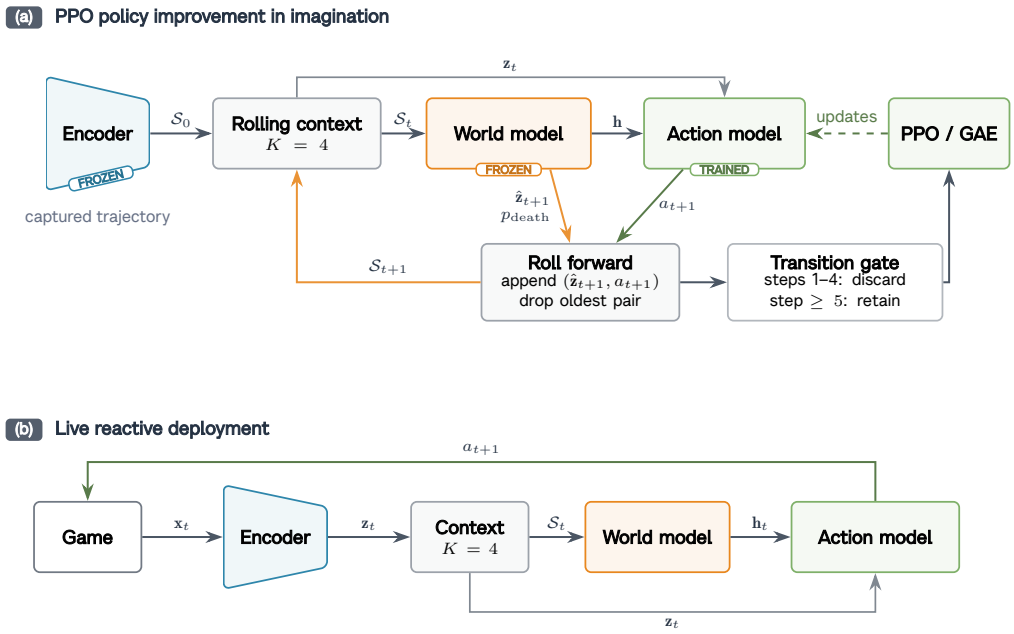

Figure 1 Learning in imagination and acting in the live game. **(a)** A recorded context seeds the frozen world model; the action model chooses what to do, the world model predicts the consequence, and PPO updates only the action model. **(b)** In the baseline live path, the encoder and temporal context provide the state used for action selection, while visual generation is skipped.

3.2 Action-conditioned dynamics

The dynamics model is an eight-layer, width-384 transformer that receives four recent frame–action pairs. Each frame block contains 64 visual tokens and one ALIVE/DEATH status token, with action tokens inserted between blocks; a final masked block requests the next frame. Attention is bidirectional within a frame and causal across frame/action blocks, so all next-frame positions are predicted in one forward pass without seeing their targets. An explicit action token separates consecutive frames, so the model can answer the question central to policy learning: what changes if the action model jumps rather than idles? It predicts all 64 codes of the next grid in parallel, together with an ALIVE/DEATH score, and exposes a temporal summary $\mathbf{h}_t$, taken from the most recent action-token position, to the action model. Parallel prediction keeps imagined steps inexpensive.

Training combines next-grid prediction with action-conditional contrastive predictive coding of weight 0.1 (Burchi and Timofte, 2025; van den Oord et al., 2018), context corruption, and an auxiliary death objective. For context corruption, 5% of context tokens are replaced uniformly and another 5% with adjacent FSQ codes. The token loss uses focal modulation (Lin et al., 2017) and structured label smoothing (SLS): for target lattice coordinate $\mathbf{c}_i$, smoothing mass $\varepsilon = 0.1$ is distributed over non-target codes proportionally to $\exp(-\|\mathbf{c}_i - \mathbf{c}_j\|_2^2/2\sigma^2)$, with $\sigma = 0.9$. Focal modulation is applied once to the aggregate soft-target cross-entropy, and the output projection is tied to the token embedding.

3.3 Controller and latent policy improvement

The 45,546-parameter actor–critic embeds the current token grid, processes it with two convolutions and max pooling, and concatenates the resulting 256-dimensional feature with the dynamics model's temporal summary $\mathbf{h}_t$. Separate heads predict jump probability, value, and an eight-action auxiliary output covering the current and next seven actions. The matched spatial-only ablation removes $\mathbf{h}_t$ from BC, PPO, and deployment, leaving a 41,706-parameter action model; the world model still generates its PPO rollouts. BC first reproduces recorded action timing; PPO then departs from imitation through repeated imagined consequences.

Each PPO iteration generates 512 imagined episodes of at most 45 steps from recorded four-frame contexts, with greedy next-grid predictions and sampled Bernoulli actions. The first four generated transitions refresh the context but are excluded because the recorded approach may make collision unavoidable; the remainder covers visible obstacles before invented geometry dominates. Rollouts terminate when the death logit exceeds the ALIVE logit; because death frames are oversampled 5×, this is an operational boundary rather than a calibrated probability. Reward is +1 per survived step and −0.25 per jump, discouraging a local optimum in which an early jump merely delays collision while airborne and earns extra survival reward without clearing the obstacle. We use Generalized Advantage Estimation (GAE) (Schulman et al., 2015) with $\gamma = 0.995$, $\lambda = 0.95$, PPO clip 0.2, and auxiliary-action loss weight 0.1.

3.4 Reactive deployment path

Training generates consequences; baseline live control only interprets recent observations. Desktop Duplication captures only the 1032×1032 model crop; CPU grayscale conversion followed by CUDA Sobel filtering and area resizing produces the 64×64 observation. Each frame is encoded and appended to a GPU-resident context, and the transformer runs context prefill only to produce $\mathbf{h}_t$. The live context marks every observation ALIVE; process-memory death detection only terminates and restarts an attempt outside the controller. The next-grid predictor and image decoder are skipped, reducing the deployed path to 15.57 million parameters; the encoder uses `torch.compile`, and transformer context encoding uses a CUDA Graph after warm-up. The accelerated preprocessing path was checked against the recorded-data pipeline on synthetic and fresh live captures and produced byte-identical observations. Spatial-only deployment loads only the frozen encoder and action model; both paths are reactive controllers whose policy improvement occurred during offline imagined training.

4 Experimental setup

Data and training. Before augmentation, the corpus contains 4,264 action-labelled gameplay episodes, or 251,963 frames and approximately 2.33 hours at the nominal 30-FPS capture cadence. It includes deliberate failures, official Levels 1–7, several community levels, and 36 expert trajectories or segments totaling less than 20 minutes. BC uses action labels from both the deliberate-failure and expert corpora. Recording samples the current key state after each capture; deployment appends the new capture with the previously held state before dispatching the next, while process memory only segments deaths and restarts. Mechanics outside the selected scope, including gravity inversion, are excluded. Failure and expert trajectories are each split 90/10 with seed 42 at the base-episode level. During tokenizer training, each adjacent-frame pair receives a shared two-dimensional translation whose horizontal and vertical offsets are sampled independently and uniformly from $[-4, 4]$ pixels. For dynamics training, the frozen encoder instead tokenizes each base episode at vertical offsets $\{-4, -2, 0, 2, 4\}$, expanding the corpus to 21,320 episode variants. All windows and variants inherit their base-episode assignment; although no complete trajectories are duplicated, deterministic repeated attempts can share near-identical prefixes, so the 10% stratum is an in-distribution development set rather than a generalization test. The tokenizer, dynamics model, and BC controller use this split; PPO draws rollout seeds from the full corpus and uses 512 fixed development contexts for checkpoint selection. Tokenizer and dynamics training use seed 42, while controller training is repeated with seeds 43–45 under the same frozen representation and world model. The first two stages run sequentially in BF16 on one A100, controller training runs in BF16 on one H200, and live inference and local latency evaluation use the same RTX 2060 with PyTorch 2.11.0+cu126 (CUDA 12.6).

The retained full runs took 3.19 A100 hours for the tokenizer, 4.20 A100 hours for the dynamics model, and 17.41 H200 hours for the seed-43 $H = 45$ controller, including BC and 15,000 PPO iterations. Thus the retained pipeline required 24.80 hours of sequential training while occupying one GPU at a time—roughly one day end to end. The matched spatial-only controller took 17.12 H200 hours under the same iteration budget.

Checkpoint and live evaluation. PPO checkpoint selection uses survival on 512 fixed latent development contexts every ten iterations. All live checkpoints are then frozen and act deterministically from pixels alone; Auto-Retry standardizes restarts but provides no policy input. For each seed, we compare PPO with the

Table 1 Training and checkpoint selection; tokenizer/dynamics learning rates use cosine decay.

Stage	Optimization	Selection
Tokenizer	Adam; 1,000 epochs; batch 2,048; LR $10^{-3} \rightarrow 10^{-5}$; slowness 0.1; uniformity 0.01.	Min. loss (epoch 920).
Dynamics	AdamW; 200×500 steps; batch 512; LR $2\times10^{-3} \rightarrow 5\times10^{-5}$; weight decay 0.01; dropout 0.1; death oversampling 5×.	Max. death F1 (epoch 139).
BC	Three seeds; AdamW; 50 epochs/seed; batch 512; LR 10^{-3}; weight decay 10^{-4}; positive jump weight 1.5.	Min. loss (10; 10; 11).
PPO	Three seeds; Adam; 15,000 iterations/seed; 512 rollouts/iteration; four update epochs; minibatch 512; LR 10^{-4}; entropy/critic weights 0.01/0.5; ratio clip 0.2; max grad. norm 0.5.	Max. survival (12,420; 5,090; 12,280).

exact BC checkpoint that initialized it over 25 scored attempts on *Stereo Madness*, *Back on Track*, and *Polargeist*. BC and PPO attempts are separate samples paired only by lineage and run until detected death. Background appearance cycles independently of level progress, while capture and dispatch phases vary near timing boundaries; repetitions measure this pixel-level deployment variability. The protocol also covers *Stereo Madness Copy*, which begins at Level 1's first ship section, and post-freeze community layout *Stereo INSANE Nerfed*, a held-out test within supported mechanics. To validate the earlier 30-FPS evaluations, a cadence check compares the identical frozen PPO checkpoint on Stereo Madness Copy over 25 optimized-path attempts at 60 FPS and 20 earlier diagnostic-path attempts at 30 FPS.

The endpoint is frames survived, except the cross-cadence probe uses wall time. PPO-minus-BC intervals independently resample each frozen policy's 25 attempts 50,000 times and characterize deployment variability; the three seed differences provide replication.

A post-freeze probe retrains seed 43 from the same BC parent with $H = 20$ for 30,000 iterations (15.61 H200 hours), versus $H = 45$ for 15,000 (17.41 hours), and scores 10 versus 25 attempts per layout. Because horizon and iteration count co-vary, this is a roughly compute-matched shorter-rollout probe, not an isolated ablation.

A second seed-43 probe keeps the $H = 45$ recipe and 15,000-iteration budget but removes $\mathbf{h}_t$ from controller training and inference. It scores the resulting BC and PPO checkpoints over 10 attempts on the three official levels and held-out layout; comparisons with the 25-attempt temporal-state reference independently bootstrap the two frozen-policy samples.

Latency and continuation protocols. Live latency covers capture through dispatch over 5,000 optimized-path frames. The qualitative intervention starts immediately before a spike from one encoded four-frame prefix; greedy branches differ only in their first action and then idle. Quantitative diagnostics re-encode the unaugmented development trajectories, yielding 22,745 next-frame windows from 413 usable trajectories. The paired intervention flips only the final context action and bootstraps trajectories 5,000 times; autoregressive diagnostics greedily feed predictions back while replaying recorded actions. The standard cohort has 1,024 starts from 153 trajectories; the exploratory 200-step cohort has 256 starts from the only four sufficiently long trajectories.

Targeted dynamics-loss ablations. We compare the reported structured-SLS+CPC dynamics model with three single-training-run variants: uniform label smoothing with the same $\varepsilon = 0.1$, no label smoothing, and structured SLS without CPC. They retain the tokenizer, corpus, split, architecture, seed 42, optimization, and maximum-development-death-F1 selection rule; all checkpoints share the exact evaluation windows and rollout starts.

External transition-latency protocol. We also time the native imagined transition of DashVMC and released Pong checkpoints of IRIS and DIAMOND on the same RTX 2060. Each runs in a fresh process at batch one with FP32 eager execution for 30 warm-ups and two repetitions of 100 synchronized transitions; differing native interfaces make this a cost, not quality, comparison.

5 Results

5.1 Policy refinement in the live game

The central question is whether policy improvement learned only through frozen-model rollouts transfers beyond imitation to the live game. The selected tokenizer reconstructs development edge maps at 34.10 dB PSNR, and the dynamics model reaches 29.58% visual-token accuracy. Deaths comprise 1.80% of its 22,745 development windows; at threshold 0.5 the death head obtains 0.721 precision, 0.883 recall, 0.794 F1, 0.994 AUROC, and 0.050 expected calibration error. Because this stratum selected the dynamics checkpoint, these are in-distribution development diagnostics rather than untouched test results. These component metrics establish a usable but imperfect imagined environment; the live comparison below tests whether it nevertheless teaches better actions.

Table 2 Live survival on all five layouts. No-op entries are mean frames ± standard deviation over 10 attempts and are shared across seeds. BC/PPO entries use 25 attempts; Δ is PPO minus its exact parent BC [95% attempt-level bootstrap interval]. The first three are official levels, Stereo Madness Copy begins in ship form, and Stereo INSANE Nerfed is held out.

Level	Policy	Seed 43	Seed 44	Seed 45
Stereo Madness	No-op		shared: 46.2 ± 0.7	
	BC	120.2 ± 91.8	130.6 ± 76.0	156.0 ± 100.2
	PPO	$\mathbf{280.1 \pm 23.3}$	$\mathbf{280.4 \pm 33.5}$	$\mathbf{308.8 \pm 68.6}$
	Δ [95% CI]	159.9 [122.7, 195.4]	149.8 [116.8, 180.6]	152.8 [106.7, 199.8]
Back on Track	No-op		shared: 63.4 ± 0.7	
	BC	123.5 ± 81.5	111.1 ± 60.3	120.2 ± 69.6
	PPO	$\mathbf{260.0 \pm 55.5}$	$\mathbf{195.9 \pm 126.5}$	$\mathbf{209.4 \pm 130.0}$
	Δ [95% CI]	136.5 [98.8, 174.4]	84.8 [31.9, 139.6]	89.2 [34.6, 147.2]
Polargeist	No-op		shared: 41.5 ± 0.7	
	BC	52.4 ± 16.7	49.0 ± 16.5	46.2 ± 9.2
	PPO	$\mathbf{65.2 \pm 33.4}$	$\mathbf{68.3 \pm 44.1}$	$\mathbf{63.1 \pm 32.9}$
	Δ [95% CI]	12.8 [−0.2, 28.6]	19.4 [3.0, 39.0]	16.8 [5.7, 31.5]
Stereo Madness Copy	No-op		shared: 135.0 ± 0.0	
	BC	366.8 ± 172.8	285.7 ± 153.5	299.4 ± 175.2
	PPO	$\mathbf{435.6 \pm 196.1}$	$\mathbf{495.8 \pm 137.5}$	$\mathbf{546.8 \pm 94.1}$
	Δ [95% CI]	68.8 [−33.3, 166.1]	210.1 [127.9, 285.5]	247.4 [167.6, 320.7]
Stereo INSANE Nerfed	No-op		shared: 46.1 ± 0.8	
	BC	104.8 ± 72.1	102.1 ± 64.0	97.8 ± 57.9
	PPO	$\mathbf{290.4 \pm 48.6}$	$\mathbf{342.8 \pm 80.8}$	$\mathbf{266.7 \pm 128.9}$
	Δ [95% CI]	185.5 [151.1, 217.6]	240.6 [200.8, 280.2]	168.8 [115.4, 223.9]

In Table 2, BC exceeds no-op everywhere and PPO improves over its parent in all fifteen seed-level comparisons. Across the three controller seeds, the paired improvements are 154.2 ± 5.2 frames on *Stereo Madness*, 103.5 ± 28.7 on *Back on Track*, and 16.3 ± 3.3 on *Polargeist*, where the variation is the sample standard deviation across the three seed differences. Every interval on the first two levels excludes zero; the smaller Polargeist gain is positive for all seeds, but seed 43's interval marginally includes zero. Polargeist's smaller gain may reflect its yellow-orb mechanic, which requires a second, precisely timed jump input while airborne and is absent from the preceding levels.

On Stereo Madness Copy, seed 43's interval includes zero but seeds 44–45 exclude it (mean gain 175.4 ± 94.2); on held-out Stereo INSANE Nerfed, all exclude zero (mean gain 198.3 ± 37.6). The Copy is not held out, but exposes strong altitude control from Level 1's first ship section. The 60-FPS cadence probe on Stereo Madness Copy yields similar observed real-time survival: mean recorded episode wall time is 17.72 s at 60 FPS versus 17.79 s at 30 FPS, with a 95% bootstrap interval of $[-1.04, 0.89]$ s for the mean difference (60 FPS minus 30 FPS). This limited one-checkpoint, one-layout probe supports cadence transfer there, not equivalence across the primary cube levels.

Runtime temporal-state ablation. The matched spatial-only controller gives up 1.11 frames on the fixed dream-selection metric, reaching 29.02/45 versus 30.13/45 for the seed-43 temporal-state reference.

Table 3 Seed-43 controller-input ablation (mean frames ± sample SD). Dream is best survival on the same 512 fixed contexts; live results use 25 attempts for the temporal-state reference and 10 for spatial-only.

Controller input	Dream	Stereo	Back	Polargeist	INSANE *Nerfed*
$\mathbf{z}_t + \mathbf{h}_t$	30.13	280.1 ± 23.3	260.0 ± 55.5	65.2 ± 33.4	290.4 ± 48.6
$\mathbf{z}_t$ only	29.02	277.9 ± 19.2	294.9 ± 64.4	128.0 ± 65.1	248.1 ± 77.3

Live performance is largely retained without $\mathbf{h}_t$. This suggests that temporal state contributes little in this predominantly reactive game, where the current grid exposes nearby geometry and avatar position and recent history mainly disambiguates motion such as scrolling and vertical velocity. A memory-dependent environment would be expected to show a larger drop when $\mathbf{h}_t$ is removed. Although $\mathbf{h}_t$ improves dream-selection survival by 1.11 frames, the world model's demonstrated role remains to provide PPO's offline training environment.

5.2 Live control at game speed

The complete perception-to-action loop must finish within 16.67 ms; at 60 Hz it can revise the binary key state every frame, not press 60 times per second. While the RTX 2060 also renders the game, 5,000 measurements have mean/median 12.018/12.083 ms, p95 14.068 ms, p99 14.913 ms, and maximum 24.059 ms. Eleven frames (0.22%) exceed the period; on a miss the loop skips its pacing sleep and immediately begins the next capture rather than reusing an old action. This doubles the temporal resolution of the 30-FPS demonstrations. Qualitatively, the ship policy uses sustained thrust and release intervals: ship motion remains correctable in flight, whereas a cube jump is largely ballistic until landing and therefore demands sparser, more precise timing. The smooth, low-duty-cycle behaviour is consistent with PPO's per-step jump penalty, although it does not isolate that penalty's effect.

5.3 Cost of an imagined step

DashVMC's native imagined transition averages 11.05 ms, compared with 49.30 ms for DIAMOND and 177.83 ms for IRIS, corresponding to 4.5× and 16.1× lower latency, respectively. Only DashVMC fits the 16.67-ms 60-FPS period on this hardware.

Table 4 Batch-1 native imagined-transition latency on an RTX 2060 (synchronized run means in milliseconds). Interfaces differ, so this compares operational cost rather than quality.

Model	Run 1	Run 2	Mean
DashVMC	11.04	11.06	**11.05**
DIAMOND	50.13	48.46	49.30
IRIS	176.94	178.73	177.83

Parallel grid prediction and latent-space feedback avoid autoregressive token generation and pixel decoding at every imagined step, compounding the savings through PPO.

5.4 Action sensitivity and autoregressive fidelity

Beyond its 45-step training horizon, the model feeds predicted grids and new actions back into context. Interactive rollouts reproduce scrolling terrain, camera and height changes, collisions, and avatar transformations; long continuations may remain coherent or eventually repeat, empty, or invent impossible geometry. Video demonstrations of interactive level continuations are available on the project page.

Table 5 Recorded-action rollout diagnostics. Accuracy and decoder-space PSNR compare predicted with recorded grids; JS compares pooled token marginals. Standard: 1,024 starts from 153 trajectories; extended: 256 starts from four long trajectories.

Cohort	Horizon	Token acc. ↑	PSNR ↑	Token JS ↓
Standard	1	30.14%	32.83	0.0037
	5	19.29%	28.25	0.0045
	10	14.06%	25.23	0.0052
	20	8.26%	22.45	0.0079
	45	2.91%	19.99	0.0208
Extended	45	3.02%	19.78	0.0351
	100	1.38%	19.49	0.0428
	200	0.74%	19.39	0.0733

Table 5 is best read relative to screen replacement: scrolling replaces the entire visible screen in roughly 45 frames. The decline from 32.83 dB at horizon 1 to 19.99 dB at 45 therefore marks a transition to fully generated geometry, not only compounding error. Thereafter PSNR is nearly flat within the extended cohort (19.78/19.49/19.39 dB at horizons 45/100/200): long rollouts can remain coherent, but beyond one screen of travel they are not reliable counterfactual environments for PPO.

Figure 2 instead isolates action sensitivity: from one fixed context, changing one action makes the jump branch clear the spike while idle collides. Across all 22,745 development transitions, the factual action has lower next-frame negative log-likelihood than its flipped counterpart in 69.52% of contexts. The trajectory-mean advantage is 0.165 nats per visual token (95% episode-bootstrap interval [0.150, 0.181]); factual-action token accuracy is 29.58% versus 27.78% after flipping, and 11.04% of argmax token predictions change.

Figure 2 Action-conditioning diagnostic from one shared four-frame context. The rows are aligned at $t+1$ and $t+5$: changing only the first future action makes the idle branch collide and terminate, while the jump branch clears the spike and lands by $t+13$.

Table 6 Matched dynamics-loss ablations on fixed development data. **Ours** uses structured SLS and CPC; other row names denote the changed component. Action advantage is the episode-mean factual-versus-flipped NLL difference. Rollout columns average horizons 1/5/10/20/45. Each row is one run; bold is best.

Variant	Death F1 ↑	Action adv. ↑	Mean acc. (%) ↑	Mean PSNR ↑	Mean JS ↓
Ours	.794	.165	**14.93**	**25.75**	**.0084**
No SLS	.808	**.196**	14.33	25.65	.0115
No CPC	.787	.148	13.89	25.56	.0104
Uniform LS	**.826**	.156	12.90	25.41	.0141

Table 6 averages each rollout curve over the five horizons; Ours leads accuracy and PSNR while minimizing token-marginal JS, and removing CPC worsens every endpoint. Removing SLS improves death F1 and factual-action advantage but reduces all three rollout aggregates, whereas uniform smoothing attains the highest death F1 but the weakest rollout profile. These single-run diagnostics support CPC and associate FSQ-structured smoothing with better rollout fidelity, without establishing training variance or downstream policy effects.

Rollout horizon. The roughly compute-matched seed-43 $H = 20$ probe changes macro-average live survival by $+1.9\%$ relative to $H = 45$, with a 95% stratified-bootstrap interval of $[-8.1, 13.5]\%$; every per-layout interval includes zero. Thus 20-step dreams preserve observed performance and show that late $H = 45$ states are not necessary for this run, but the single seed, unequal attempts, and co-varying iteration count preclude an isolated horizon claim.

6 Limitations

DashVMC studies one deterministic binary-action game, with one post-freeze layout within known mechanics. Repeated level prefixes make model diagnostics in-distribution; three controller seeds share one tokenizer and dynamics checkpoint, measuring controller rather than end-to-end uncertainty. A matched seed-43 spatial-only probe finds no consistent live deficit after removing $\mathbf{h}_t$, but its 10-attempt comparison does not establish equivalence; no direct frame-stack controller isolates the value of discrete visual tokens. Death calibration covers only recorded states, leaving recursive reward exploitation unquantified. Loss ablations are single-run model diagnostics; controller-input and $H = 20$ probes are single-seed, and the latter is only roughly compute matched. One-GPU access constrained multi-seed ablations and scaling curves, while wall-clock live evaluation limited deployment tests. Finally, 60-FPS behavioural equivalence is probed on one checkpoint and layout, and reported speed depends on the RTX 2060 and compact edge representation.

7 Conclusion

DashVMC turns a fixed archive of roughly two hours of action-labelled gameplay into an interactive, action-conditioned predictive model of the game. Its latent rollouts form a learned environment for offline model-based reinforcement learning: PPO improves BC-initialized controllers without additional live interaction, and the resulting policies consistently outlive their BC parents after zero-shot transfer to the non-paused game. The deployed pipeline meets the game's 60-Hz timing budget, while the spatial-only ablation shows that the learned dynamics can enable policy improvement without remaining in the live control path. Under data scarcity, the world model thus acts as an interface between past experience and future decisions without prescribing how it must be used: DashVMC uses that interface for imagined policy optimization, while planning-based agents could instead query it at evaluation time.

Acknowledgments

We thank Clément Chatelain and Robin Condat for their guidance during the Representation Learning course at INSA Rouen Normandy, in which this project began, and CRIANN for providing the computing resources used for training. We thank Maël Planchot for contributing gameplay data. Geometry Dash is developed by RobTop Games.

References

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (VPT): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.

Maxime Burchi and Radu Timofte. TWISTER: Learning transformer-based world models with contrastive predictive coding. In *International Conference on Learning Representations*, 2025.

David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.

Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Gabriel Dulac-Arnold, Ian Osband, John Agapiou, Joel Z. Leibo, and Audrunas Gruslys. Deep q-learning from demonstrations. In *AAAI Conference on Artificial Intelligence*, 2018.

Yann LeCun. A path towards autonomous machine intelligence. *OpenReview*, 2022. Version 0.9.2.

Ted Li and Sean Rafferty. Playing Geometry Dash with convolutional neural networks. Stanford CS231N course report, 2017.

Bin Lin, Zongjian Li, Yuwei Niu, Kaixiong Gong, Yunyang Ge, Yunlong Lin, Mingzhe Zheng, JianWei Zhang, Miles Yang, Zhao Zhong, Liefeng Bo, and Li Yuan. iFSQ: Improving FSQ for image generation with 1 line of code. *arXiv preprint arXiv:2601.17124*, 2026.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *International Conference on Computer Vision*, pages 2980–2988, 2017.

Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.

Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *International Conference on Machine Learning*, 2024.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.

Zaishuo Xia, Yukuan Lu, Xinyi Li, Yifan Xu, and Yubei Chen. Clone deterministic 3d worlds with geometrically-regularized world models. *arXiv preprint arXiv:2510.26782*, 2025.